

How a Rupee Moves Through the UPI Simulator

A complete beginner-to-architecture explanation of the services, HTTP requests, routing, bank ledgers, settlement, reversal, and the real-world UPI/NPCI mental model.

Core lesson: a payment is not one HTTP request travelling through every service. Each service receives a request, performs its responsibility, and may create a new request to the next service.

Service	Port	Role in the simulator
Frontend / Browser	3000	Collects payment details and starts the payment
PSP	5001	Creates transaction ID and orchestrates the flow
NPCI Switch	5002	Routes the transaction to the correct bank services
Bank A	5003	Owns one bank ledger and exposes debit/credit operations
Bank B	5004	Owns another bank ledger and exposes debit/credit operations
Auth	5005	Simplified authentication service in the simulator
Ops Console	5010	Read-only operational/ledger view

Amounts are represented as integer paise in the simulator. For example, Rs.500 = 50,000 paise.

1. The most important concept: request chaining

Suppose the user pays Rs.500 from you@mockbank to swiggy@mockbank2. The browser sends an HTTP request to the PSP. The PSP then creates a new HTTP request to the switch. The switch then creates new requests to the relevant bank services.

```
Browser :3000
|
| POST /initiate
| { from, to, amount }
v
PSP :5001
|
| POST /route
| { txn_id, from, to, amount }
v
NPCI Switch :5002
|
+---- POST /debit -----> Payer Bank
|
+---- POST /credit -----> Payee Bank
```

So 5001 -> 5002 is not OS-level port forwarding. It is service A making a new HTTP request to service B.

The response then travels back through the relevant caller. This is why the application can be understood as a chain of request/response interactions.

2. What is inside the first request: 3000 -> 5001

The frontend has the information entered by the user. Conceptually the request is:

```
POST http://localhost:5001/initiate
```

Content-Type: application/json

```
{
  "from": "you@mockbank",
  "to": "swiggy@mockbank2",
  "amount": 500
}
```

The PSP receives this as req.body after JSON parsing. At this stage there may be no transaction ID yet; the PSP creates one.

```
req.body
{
  from: "you@mockbank",
  to: "swiggy@mockbank2",
  amount: 500
}
```

PSP creates:
txn_id = "TXN123"

3. 5001 -> 5002: PSP to NPCI switch

The PSP takes the payment details, adds the transaction ID, and creates another HTTP request.

POST http://localhost:5002/route
Content-Type: application/json

```
{
  "txn_id": "TXN123",
  "from": "you@mockbank",
  "to": "swiggy@mockbank2",
  "amount": 500
}
```

The switch now has enough information to resolve the two handles to their respective bank participants/services.

```
from = you@mockbank
      |
      +--> Bank A

to = swiggy@mockbank2
    |
    +--> Bank B
```

4. 5002 -> 5005: authentication

In the simulator, the switch/flow invokes the separate authentication service before the debit. Conceptually:

POST http://localhost:5005/step-up/verify
Content-Type: application/json

```
{
  "txn_id": "TXN123",
  "account": "you@mockbank",
  "method": "pin",
  "credential": "...
}
```

The authentication service checks the required credential/authentication data and returns a result such as verified=true. The important architecture lesson is that authentication is a separate service in this simulator.

Real-world nuance: real UPI does not require a separate generic auth microservice like this simulator. NPCI's published UPI material describes the remitter bank as responsible for authenticating the UPI PIN. This simulator simplifies that responsibility into port 5005.

5. The critical correction: 5003 and 5004 are banks, not roles

Do not memorize '5003 = payer' and '5004 = payee'. That would be wrong as a general architecture.

5003 = Bank A service

5004 = Bank B service

Each bank can be:

- payer/remitter bank for one transaction
- payee/beneficiary bank for another transaction

The operations are what have fixed meaning: /debit means the receiving bank should debit the specified payer account; /credit means the receiving bank should credit the specified beneficiary account.

Therefore, ideally both bank services expose both operations:

Bank A :5003

```
POST /debit
POST /credit
POST /reverse
POST /settlement/post
GET /txn/:id
GET /all-transactions
```

Bank B :5004

```
POST /debit
POST /credit
POST /reverse
POST /settlement/post
GET /txn/:id
GET /all-transactions
```

6. 5002 -> payer bank: /debit

For KUR/HDFC -> Swiggy/XYZ, suppose HDFC is represented by :5003 and XYZ by :5004.

5002

```
|
| POST /debit
| {
|   txn_id: "TXN123",
|   account: "KUR@HDFC",
|   amount: 500
| }
```

v

HDFC service :5003

|

v

HDFC ledger database

The bank service, not NPCI, performs the ledger operation. In the simulator it creates a balanced journal entry and postings.

```
DR KUR@HDFC    Rs.500
CR HDFC:POOL   Rs.500
```

The service then returns a result such as debited=true/status=debited to the caller.

7. 5002 -> payee bank: /credit

After the debit succeeds, the switch sends a new request to the beneficiary bank.

```
5002
|
| POST /credit
| {
|   txn_id: "TXN123",
|   account: "swiggy@XYZ",
|   amount: 500
| }
```

```
v
XYZ service :5004
```

```
|
v
XYZ ledger database
```

```
DR XYZ:P00L          Rs.500
CR swiggy@XYZ        Rs.500
```

Again, the bank service modifies its own database. The switch does not directly open or write the other bank's SQLite file.

8. What happens when direction reverses?

Tomorrow Swiggy can pay KUR. Nothing about the bank services needs to become a permanent 'payer server' or 'payee server'.

```
Swiggy@XYZ ---- Rs.1,000 ----> KUR@HDFC
```

```
Payer bank  = XYZ   :5004
Payee bank  = HDFC  :5003
```

```
5002
|
+---- POST /debit  -----> 5004 (XYZ)
|
+---- POST /credit -----> 5003 (HDFC)
```

The same endpoints are used, but the destination bank services are reversed because the transaction roles are reversed.

9. What if the payer is SBI?

This is where the limitation of a two-bank simulator becomes obvious. If your simulator only has HDFC :5003 and XYZ :5004, there is no SBI participant to receive a debit request.

A more scalable simulator would model bank identity separately from transaction role.

Bank routing registry

```
HDFC -> http://localhost:5003
XYZ   -> http://localhost:5004
SBI   -> http://localhost:5006
ICICI -> http://localhost:5007
...
```

Then for SBI -> Swiggy/XYZ:

```
5002
|
+---- POST /debit  -----> SBI :5006
|
+---- POST /credit -----> XYZ :5004
```

For Swiggy/XYZ -> SBI:

```
5002
|
+----- POST /debit -----> XYZ :5004
|
+----- POST /credit -----> SBI :5006
```

This is the clean mental model: bank identities are stable; payer/payee roles are calculated for each transaction.

10. What each bank service actually does

Component	What it should do
:5003 Bank A	Receive bank operations, validate account/amount, perform ledger transaction in Bank A DB, return outcome
:5004 Bank B	Same responsibilities for Bank B
:5002 Switch	Resolve payer/payee participants, route messages, coordinate responses, manage transaction state in the simulator
:5001 PSP	Receive app payment request, create transaction ID, orchestrate the payment flow/status
:5005 Auth	Simplified authentication component in this simulator

The bank service owns its database. It should not read another bank's database to decide how to update the other bank.

11. The database model in your simulator

Each bank keeps its own ledger database. The main tables described by your simulator are:

Table	Purpose
ledger_accounts	Chart of accounts, including customers and bank accounts such as POOL, NOSTRO, SUSPENSE and OPENING
journal_entries	One row per money event; includes event type, transaction reference and reversal linkage
postings	Actual DR/CR money rows and amount in paise
holds	Funds earmarked during an in-progress debit
accounts	Identity plus cached balance; cache is not the source of truth
transactions	Payment-level status, settlement batch and failure information

The core accounting idea is append-only journal entries plus balanced postings. A cached account balance can be refreshed for convenience, but the journal remains the source of truth.

12. The Rs.500 payment end-to-end

1. Browser :3000
POST /initiate
{from, to, amount}
2. PSP :5001
creates txn_id = TXN123
3. PSP :5001 -> Switch :5002
POST /route
{txn_id, from, to, amount}
4. Authentication
Switch/flow -> Auth :5005

```

POST /step-up/verify

5. Switch :5002 -> Payer bank
  POST /debit
    {txn_id, payer, amount}

6. Payer bank DB
  DR payer
  CR payer-bank P00L

7. Switch :5002 -> Payee bank
  POST /credit
    {txn_id, payee, amount}

8. Payee bank DB
  DR payee-bank P00L
  CR payee

9. Browser -> PSP
  GET /txn/TXN123
  until terminal status

```

The request path and database ownership are separate concepts. The switch routes messages; each bank performs its own local accounting.

13. Settlement: moving pool positions to NOSTRO

The simulator later settles completed, matched transactions. The switch collects completed transaction information from both banks, matches by transaction ID, and tells each bank to post its settlement entry.

```

Remitter bank:
  DR P00L
  CR NOSTRO

```

```

Beneficiary bank:
  DR NOSTRO
  CR P00L

```

The important safety rule is that only transactions whose debit and credit legs both completed should be settled. An unmatched debit remains recoverable in the pool instead of being swept away prematurely.

The simulator also mirrors settlement information into Postgres for reporting. That reporting mirror is not the decision-making ledger.

14. Refund / reversal when the credit leg fails

If the payer was debited but the beneficiary bank did not successfully credit the recipient, the value remains in the remitter bank's pool. The system must not blindly reverse just because a network call timed out.

```

Credit leg fails / is uncertain
  |
  v
GET beneficiary transaction status
  |
  +-----+
  |               |
  | credit landed | | credit absent |
  |               | |               |
  | mark success  | | reverse      |
  |               | |               |

```

```

      v
POST /reverse
  |
DR P00L
CR payer

```

A timeout is not automatically equivalent to a failed credit. Reversing a credit that actually succeeded could create money. The simulator therefore uses status confirmation before reversal and retries when the beneficiary bank is unreachable.

The reversal is a new journal entry linked to the original entry; the original debit is not deleted.

15. The four invariants

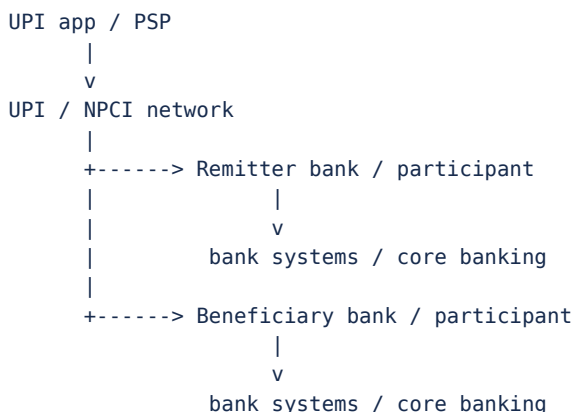
Invariant	Meaning
Every entry balances	$\text{SUM}(\text{DR}) = \text{SUM}(\text{CR})$ for every journal entry
Each bank's books balance alone	No journal entry spans two bank databases
Pools cancel	Remitter pool + beneficiary pool = 0 across the two banks
NOSTRO nets to zero after settlement	The paying bank's outflow matches the receiving bank's settlement inflow

These invariants are the reason the simulator can survive partial completion without simply changing or deleting historical records.

16. How this maps to real UPI/NPCI

The simulator is a simplified model of a real payment network. In the real world, there are participant institutions, secured network interfaces, routing/configuration, bank/PSP systems and core banking systems. You should not imagine public localhost-style endpoints.

Real-world mental model



The architectural analogy is still useful: the network routes the transaction message to the appropriate participants, and the banks perform the actual account operations in their own systems.

For a payment from SBI to HDFC, the remitter-side bank handles the debit and the beneficiary-side bank handles the credit. If the direction reverses, the same banks swap transaction roles.

17. Important real-world nuances

1. NPCI is not simply calling public REST URLs like localhost:5003. Production participant integrations use secured institutional interfaces and network/security arrangements.

2. The exact internal endpoints, protocols, routing configuration and bank architecture are not equivalent to the simplified HTTP endpoints in this simulator.
3. Authentication is simplified in the simulator. NPCI's UPI documentation describes the remitter bank as responsible for UPI PIN authentication; it is not simply a generic NPCI-owned auth microservice.
4. NPCI does not directly write into SBI or HDFC customer databases. The participating bank's own systems perform the account/ledger operation.
5. The words payer/remitter and payee/beneficiary describe roles in a particular transaction, not permanent identities of a service.

18. Final cheat sheet - remember this

BANK IDENTITY vs TRANSACTION ROLE

Bank A :5003 = Bank A
Bank B :5004 = Bank B

For transaction #1:
Bank A may be PAYER
Bank B may be PAYEE

For transaction #2:
Bank B may be PAYER
Bank A may be PAYEE

The switch determines this dynamically.

OPERATIONS

PAYER BANK -> POST /debit
PAYEE BANK -> POST /credit

BANK SERVICE

```
receives request
  |
  v
validates account / amount / state
  |
  v
opens local DB transaction
  |
  v
writes journal + postings + transaction state
  |
  v
commits
  |
  v
returns success/failure
```

NEVER think:
5003 = payer forever
5004 = payee forever

THINK:
5003 = Bank A
5004 = Bank B

Then for every transaction:
from handle -> identify payer bank
to handle -> identify payee bank

payer bank -> /debit
payee bank -> /credit

If you remember only one sentence, remember: the switch decides which bank service gets the debit and which gets the credit; each bank then performs the actual ledger operation inside its own database.